Operating Systems and Parallel Computing

Lecture 7

Prof. Matteo Turilli

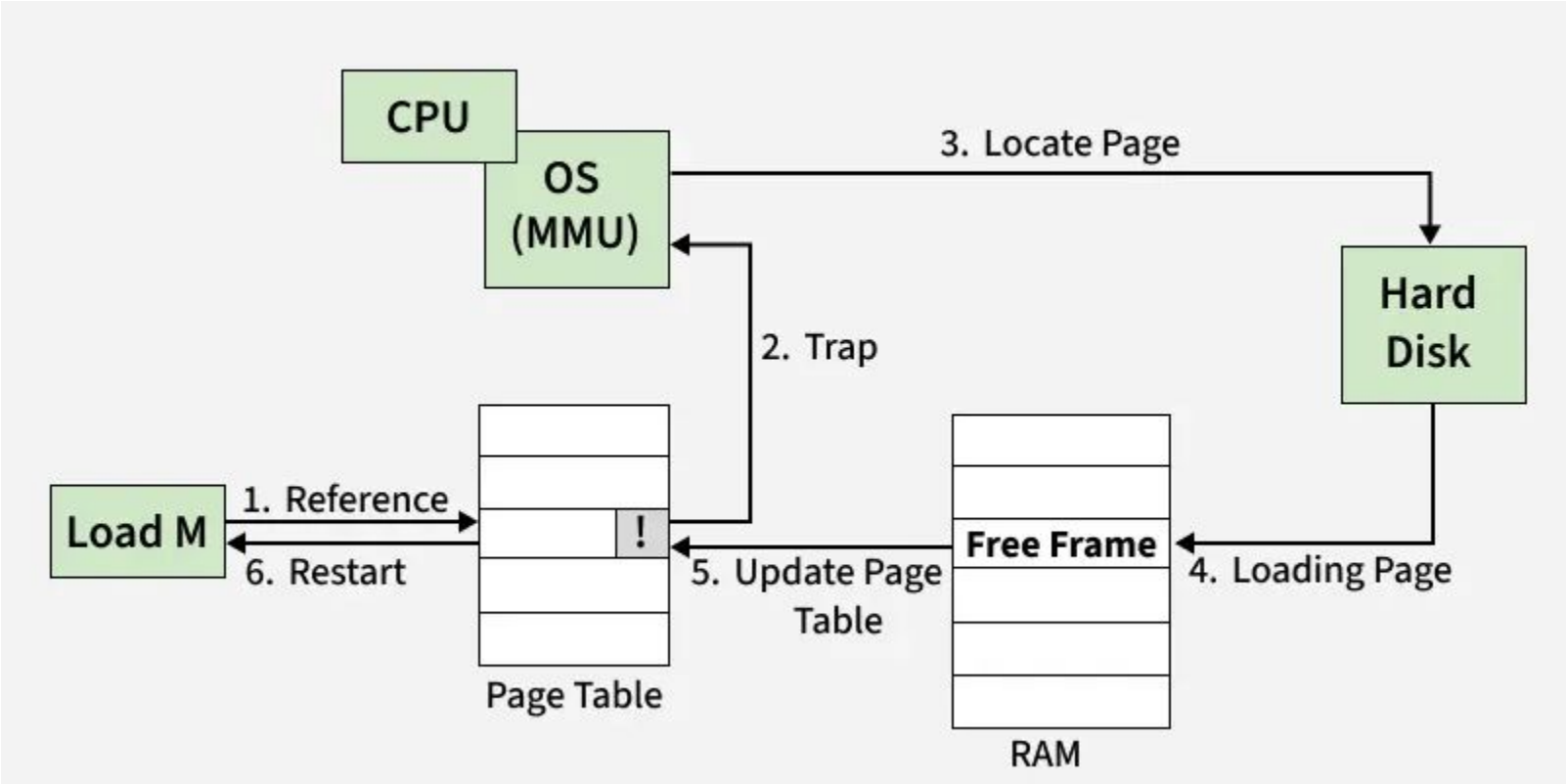
Recap

- Each process requests an amount of memory and the OS loads into RAM only the pages that are actually in use.
- **Process point of view**: it owns all the memory it needs
- **OS point of view**: for each process tracks used/unused memory pages and keeps the mapping between memory pages, physical memory frames, and swap.
- **Htop VIRT**: each process allocated memory but not necessarily used, i.e., the physical frames the OS ‘promised’ to each process.
- **Htop RES**: each process used (i.e., resident) memory, i.e., the actual physical frames the OS has handed over to each process.
- The OS (Linux) an **optimistic gambler** so it can promise more memory than the machine has, betting processes will not use all of it at the same time.
- **If the OS loses the bet, it initiates swapping; if it is not enough it starts trashing; and if that also fails, then runs the OOM killer.**

Page Fault: Asking For a Memory Page

- **Page fault**: type of hardware interrupt or exception raised by the Memory Management Unit (MMU) when a program tries to access a memory page that is not available or cannot be accessed.
- What causes a page fault:
  a. **Demand Paging**: The operating system loads pages into RAM only when requested, meaning initial access to a new page naturally triggers a fault.
  b. **Swapped Out Data**: The required data was previously moved from RAM to secondary storage (disk or swap space) to free up space.
  c. **Invalid Access**: The program requests a memory address it has no permission to access or that does not exist, causing a fatal error.
- Page faults can be soft, hard, or invalid:
  a. **Soft**: The requested data is already in RAM but is not marked in the specific process' page table or resides in a standby list; the OS just updates the mapping.
  b. **Hard**: The requested data is not in RAM and must be fetched from secondary storage (e.g., disk I/O), which takes significantly longer.
  c. **Invalid**: The memory address is illegal or unauthorized, resulting in a segmentation fault.

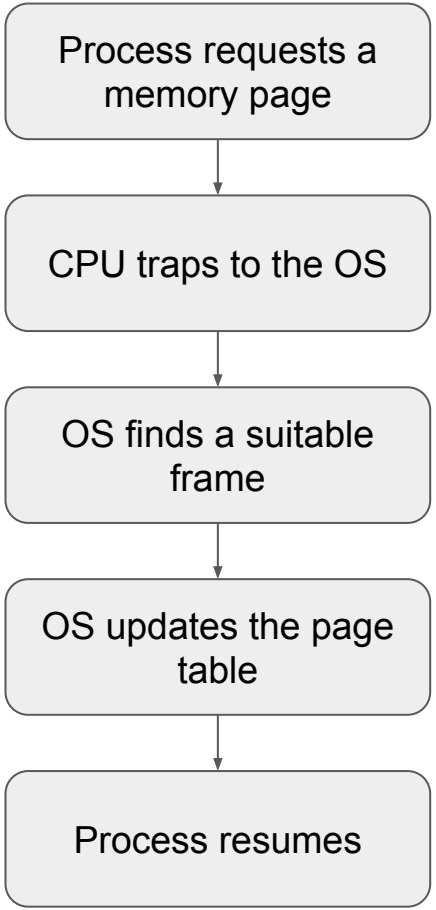
Virtual Memory System: Page Fault



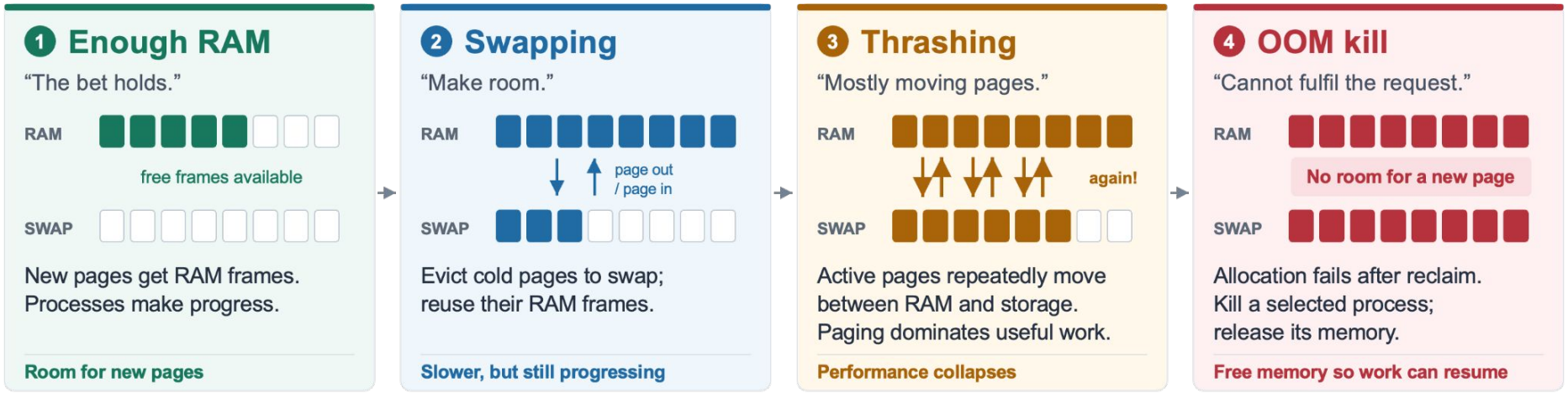
Hard page fault process

The OS Wins Its Bet

- The memory page address is in the page table, but no frame is mapped to it yet
- The hardware cannot resolve it so kernel mode takes over (see the notion of trap in Session 5)
- The OS zeroes the frame, or loads the data from disk if the content of the page was parked there
- This maps the memory page to the frame in the system’s RAM
- From the process point of view, the request for a memory page has been satisfied



The OS Loses Its Bet



- The diagram shows a possible escalation path but not the only one.
- The OS triggers the OOM killer when a valid memory page request cannot be satisfied after attempting to reclaim memory.

Page Eviction and Replacement

- Under memory pressure, the OS evict pages from RAM and reuse their frames, while keeping the pages in their process's virtual address spaces.
- E.g., the OS discards
  a. Unused file-backed pages and reload them from their file when needed.
  b. Unused pages from swap (if available).
- The OS evict a page that will not be needed soon. Accessing an evicted page causes a page fault and requires reloading its contents, i.e., it costs time!
- LRU (Least Recently Used) policy: pages not accessed recently are less likely to be accessed soon.

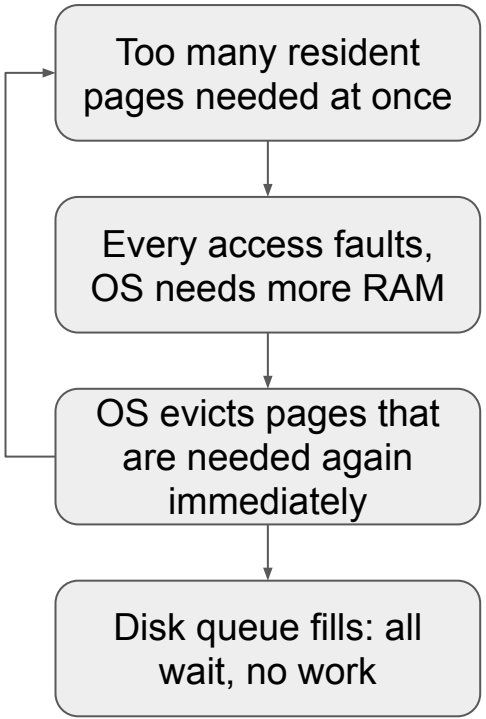
Swapping



- The OS moves cold, infrequently accessed pages to swap and reuse their physical frames. This can happen in the background, before a process needs more memory.
- Storage I/O is expensive: servicing a fault can take ms with a mechanical disk. The actual delay depends on the storage device, its load, and the I/O required.
- Occasional swapping can work well: moves rarely used pages out of RAM with little impact if the evicted pages are not needed again soon.
- Question: put your performance hat on. What if almost every memory access faults?

Thrashing

- Condition in which the performance of the OS and all the processes deteriorates to unusable levels
- The CPU looks 100% busy in htop, but the CPU is actually idling
- Kernel + I/O wait: pages moving, no work done.
- The cursor freezes, the UI freezes, the whole machine, not just one process crawls to a halt.
- Escape routes: kill something, add RAM, or need fewer pages at once.
- Modern OSs may offer the users a way to choose which application to kill (e.g., macOS)



Out of Memory (OOM) Killer in Linux



- Activation: Triggered when fail to free up memory.
- Goal: Sacrifices offending process to rescue system stability.
- The process that triggers the OOM is not necessarily the process that gets killed
- Computes a per-process "badness" score based on how much memory or swap a process uses of its allocation.
- Userspace can bias that score to make a process ineligible.
- The OOM victim is not necessarily the process that triggered the fatal allocation, not the single biggest memory user.
- Our demo scripts (`07_oom_demo.py`, `07_oom_multi_demo.py`, `07_oom_stall_demo.py`) show the basic kill, choice three competing processes of different sizes and protection levels, and a OOM failures with system freeze.

Linux vs. Windows/macOS: Two Different Bets

- Windows and macOS generally avoid Linux-style aggressive overcommit
- **Windows** imposes a memory request limit, and an exceeding request fails not allowing the process to start so no innocent bystander gets killed later.
- **macOS** leans on memory compression and swap first, but does have a OOM analogue called jetsam
- Jetsam picks processes by priority type (background/idle apps first, foreground apps last) rather than a single computed score
- Neither OS exposes an "opt out of being killed" option as Linux.







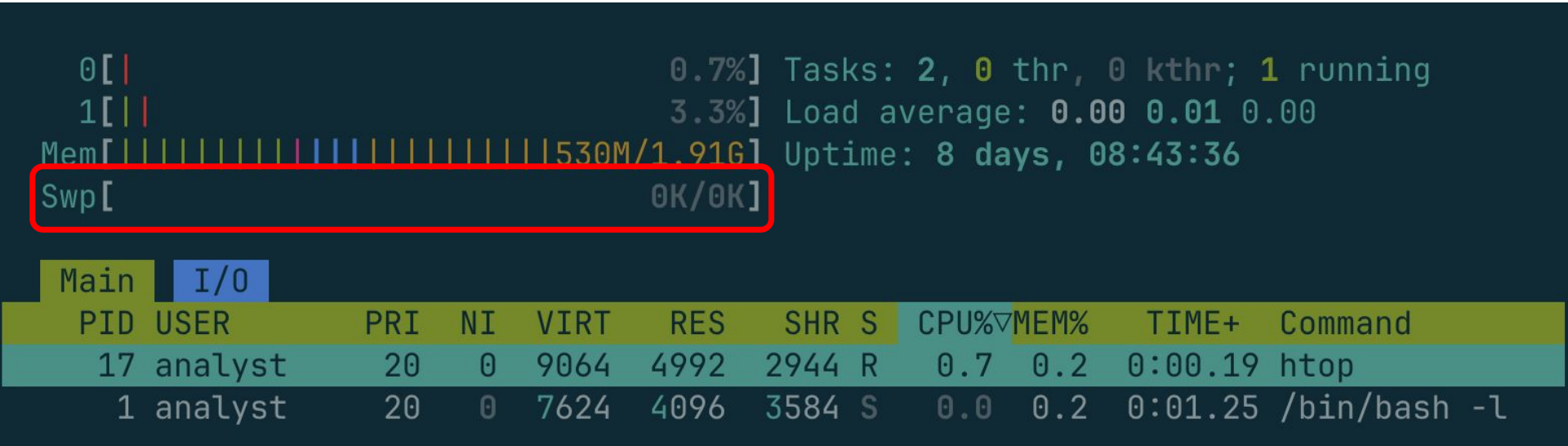


Linux vs. Windows/macOS: Different Tradeoffs

- Linux's overcommit-and-kill-later model maximizes memory sharing across many processes, at the cost of unpredictability unless someone actively tunes priorities.
- Windows/macOS's conservative accounting gives predictable, local failures and a better single-user experience, at the cost of leaving some memory capacity unused.
- Context decides the winner:
  a. Linux's approach fits multi-tenant servers and container platforms
  b. Windows/macOS model fits interactive desktops where an allocation simply returning an error is better than a random background app dying.
- Neither is "better", each optimizes for what it’s typical workload needs most: memory utilization versus predictability.

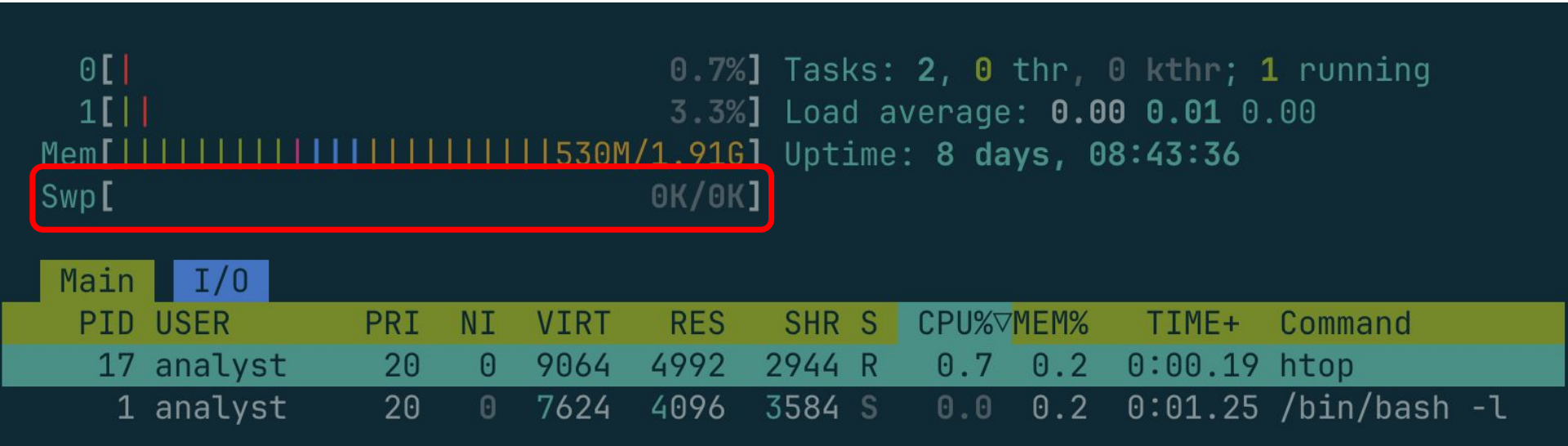
Our Container

- By default, our container does not have any swap space (see screenshot below)
- **Question**: what does a system with no swap do when RAM runs out?



Using Our Container To Study The MMU

- By default, our container does not have any swap space (see screenshot below)
- **Question**: what does a system with no swap do when RAM runs out?
- Without swap, the OS has fewer ways to free RAM. When reclaiming memory is no longer enough to satisfy a new allocation, it may kill a process.



This is an interactive class

- Download `lecture_7.zip` from Blackboard, move it into `work`, and unzip it
- Open two terminals:
  - In one run our container with `docker compose run --rm ospc`
  - In the other execute `docker ps`, find the ID of the container you just started, and copy the ID
  - Then run `docker exec -it <your_container_id> /bin/bash` and then `htop`

- Start JupyterLab: `docker compose up jupyter`
- Copy the `http://localhost:8888/lab?token=...` link into your browser.
- Download `07_memory.ipynb` from our Blackboard and move them into the `work` directory.

Exercises and Homework

- In the container, in `06_memory.ipynb`:
  - Run the notebook. What is your shallow-vs-deep ratio, and your dtype saving?
  - Add a column of your own (e.g. df["note"] = "ok") and measure it deep. Why is such a short string so expensive?
- In a terminal:
  - Run htop beside the notebook. Watch VIRT and RES as you run the `np.zeros` cell.
  - `cat /proc/meminfo | head -3` : how much memory does the container think it has? Compare with our 2 GB limit.
- Remember: VIRT is a promise, RES is reality, and dtypes are free performance.
- Next time: what happens when the promises come due and there are no free pages left? Swapping, thrashing, and the process that gets killed.